


## PIMPRI CHINCHWAD EDUCATION TRUST'S

# PIMPRI CHINCHWAD COLLEGE OF ENGINEERING

*NBA Accredited | NAAC Accredited with 'A' Grade | An Autonomous Institute*
*AICTE Approved | Permanently Affiliated to SPPU, Pune*

## Department of Information Technology

**Academic Year: 2026-2027**

# Assignment No. 3: Fractional Knapsack Problem

| | | |
|---|---|---|
| **Name** | : | **Om Rajendra Bharambe** |
| **PRN** | : | 124B1F001 |
| **Branch** | : | Information Technology (IT) |
| **Semester** | : | V |
| **Batch** | : | A1 |
| **Course Code** | : | BIT25PC04 |
| **Course Name** | : | **Design & Analysis of Algorithms Laboratory** |
| **Guided By** | : | Dr. Rajesh N. Phursule |
| **GitHub Repository** | : | om29dev/DAA_Lab_Submission |
| **Assignment Link** | : | Assignment 3 on GitHub |

# Assignment No. 3: Fractional Knapsack Problem

## 1. Problem Statement:

Emergency Relief Supply Distribution: A devastating flood has hit multiple villages in a remote area, and the government, along with NGOs, is organizing an emergency relief operation. A rescue team has a limited-capacity boat that can carry a maximum weight of W kilograms. The boat must transport critical supplies, including food, medicine, and drinking water, from a relief center to the affected villages.

Each type of relief item has:

- A weight ($w_i$) in kilograms.
- Utility value ($v_i$) indicating its importance (e.g., medicine has higher value than food).
- Some items can be divided into smaller portions (e.g., food and water), while others must be taken as a whole (e.g., medical kits).

## 2. Goals:

**1.** Implement the Fractional Knapsack algorithm to maximize the total utility value of the supplies transported.

**2.** Prioritize high-value items while considering weight constraints.

**3.** Allow partial selection of divisible items (e.g., carrying a fraction of food packets).

**4.** Ensure that the boat carries the most critical supplies given its weight limit W.

## 3. Course Objectives:

3.1. To know the basics of computational complexity of various algorithms.

3.2. To select appropriate algorithm design strategies to solve real-world problems.

## 4. Course Outcomes:

*After learning the course, students will be able to:*

4.1. Analyze the asymptotic performance of algorithms.

4.2. Solve computational problems by applying suitable paradigms such as Greedy method.

## 5. Theory:

The Fractional Knapsack Problem is a classic optimization problem: given a set of items—each with weight $w_i$ and value $v_i$—and a maximum capacity W, the objective is to maximize total value by selecting items (or fractions of them) up to capacity.

Unlike the 0/1 Knapsack, the fractional version allows partial selection, making it solvable optimally in polynomial time.

### Algorithm (Greedy Strategy):

**1.** Compute the value-to-weight ratio $v_i / w_i$ for each item.

**2.** Sort items in descending order of this ratio.

**3.** Fill the knapsack:

- Take items fully if they fit.
- If capacity runs out, take the exact fraction needed of the current item (only if divisible).
- Skip indivisible items that do not fit.

### Working of Greedy:

**1. Greedy-choice property:** Choosing the highest ratio item at each step is always part of an optimal solution.

**2. Optimal substructure:** Once part of the knapsack is filled, the remaining capacity forms a smaller instance of the same problem.

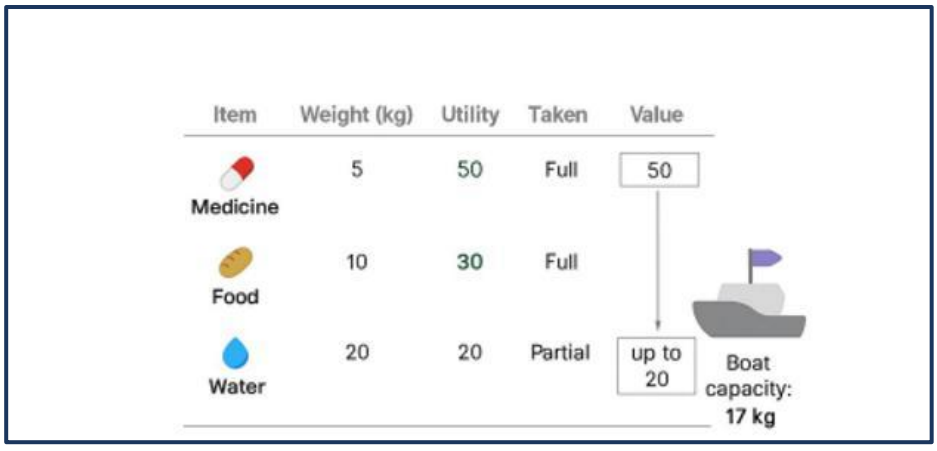



*Figure 5.1: Priority-based Item Selection and Boat Capacity Allocation*

## 6. Flowchart:

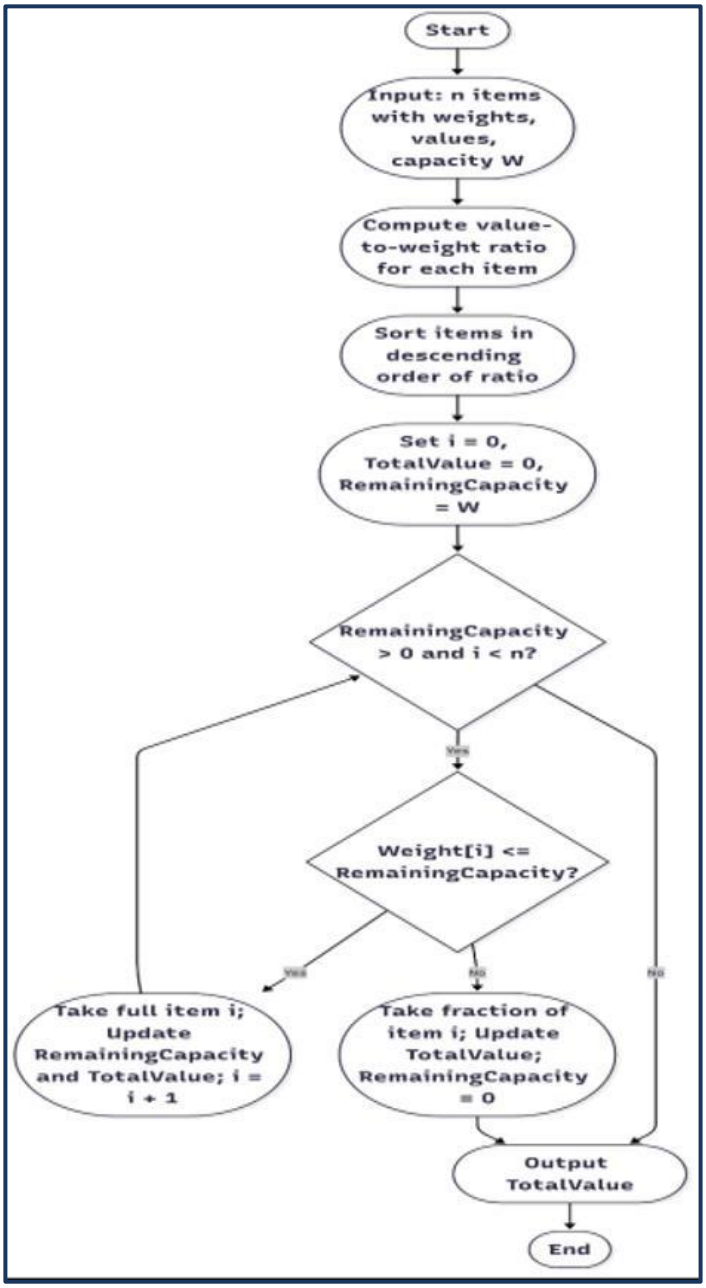



*Figure 6.1: Flowchart of Fractional Knapsack Greedy Strategy*

## 7. Time Complexity:

| Operation | Time Complexity | Explanation |
|---|---|---|
| **Ratio computation** | O(n) | Calculating value-to-weight ratio ($v_i$ / $w_i$) for all n items |

| | | |
|---|---|---|
| **Sorting** | O(n log n) | Sorting items in descending order of their ratios |
| **Selection loop** | O(n) | Iterating through items and placing into knapsack |
| **Total Time Complexity** | O(n log n) | Dominated by the sorting phase |

## 8. Pseudo Code for Fractional Knapsack:

```
function MaxUtilFractional(items, W):
  Input: items = list of (weight w, value v, isDivisible), capacity W
  Output: maximum total utility value

  Compute ratio r[i] = v[i] / w[i] for each item
  Sort items by descending r[i]
  totalValue = 0
  remainingCapacity = W

  for each item in sorted items:
    if remainingCapacity == 0:
      break
    if item.weight <= remainingCapacity:
      totalValue += item.value
      remainingCapacity -= item.weight
    else if item.isDivisible:
      fraction = remainingCapacity / item.weight
      totalValue += fraction * item.value
      remainingCapacity = 0
    // else if indivisible and can't fit, skip
  end for

  return totalValue
end function
```

## 9. Sample Example & Step-by-Step Allocation:

Boat capacity: W = 17 kg. Total potential utility if all fit = 100.

| Item | Weight (kg) | Utility | Divisible | Taken | Value Obtained |
|---|---|---|---|---|---|
| **Medicine** | 5 | 50 | No | Full | 50 |

| | | | | | |
|---|---|---|---|---|---|
| **Food** | 10 | 30 | Yes | Full | 30 |
| **Water** | 20 | 20 | Yes | Partial (if needed) | up to 20 |

### Step 1: Calculate utility per kg (Ratio $v_i$ / $w_i$):

- **Medicine:** 50 / 5 = 10 utility/kg
- **Food:** 30 / 10 = 3 utility/kg
- **Water:** 20 / 20 = 1 utility/kg

Preference order: Medicine → Food → Water

### Step 2: Fill the boat:

- **Take Medicine fully:** Weight used = 5 kg, Utility = 50, Remaining capacity = 17 - 5 = 12 kg
- **Take Food fully:** Weight = 10 kg, divisible. Fits completely. Utility = 30, Remaining capacity = 12 - 10 = 2 kg
- **Take Water partially:** Weight = 20 kg, divisible. Only 2 kg fits. Utility = 20 × (2 / 20) = 2. Remaining capacity = 0 kg.

### Step 3: Total Utility Obtained:

- **Medicine:** 50
- **Food:** 30
- **Water (partial):** 2

**Total Utility = 82**

## 10. Code and Output:

**Repository:** https://github.com/om29dev/DAA_Lab_Submission

**Assignment 3 Folder:** https://github.com/om29dev/DAA_Lab_Submission/tree/main/Assignment 3

### 10.1 C++ Implementation Source Code (emergency_relief_knapsack.cpp):

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <iomanip>
#include <string>

using namespace std;
```

```
struct Item {
    string name;
    double weight;
    double value;
    bool divisible;
    int priority;

    Item(string n, double w, double v, bool d, int p)
        : name(n), weight(w), value(v), divisible(d), priority(p) {}

    double valuePerWeight() const {
        return value / weight;
    }
};

bool compare(const Item& a, const Item& b) {
    if (a.priority != b.priority)
        return a.priority < b.priority;
    return a.valuePerWeight() > b.valuePerWeight();
}

double fractionalKnapsack(vector<Item>& items, double capacity, double&
totalWeight) {
    sort(items.begin(), items.end(), compare);

    double totalValue = 0;
    totalWeight = 0;
    int selectedCount = 0;

    cout << "\n========== SELECTED RELIEF ITEMS ==========\n";

    for (const auto& item : items) {
        if (capacity <= 0)
            break;

        double takenWeight = 0;
        double takenValue = 0;

        if (item.divisible) {
            takenWeight = min(item.weight, capacity);
            takenValue = item.valuePerWeight() * takenWeight;
        }
        else if (item.weight <= capacity) {
```

```
            takenWeight = item.weight;
            takenValue = item.value;
        }

        if (takenWeight > 0) {
            totalValue += takenValue;
            totalWeight += takenWeight;
            capacity -= takenWeight;
            selectedCount++;

            cout << "\nItem           : " << item.name;
            cout << "\nPriority       : " << item.priority;
            cout << "\nWeight Taken   : " << fixed << setprecision(2)
                 << takenWeight << " kg";
            cout << "\nUtility Value  : " << takenValue << " units";
            cout << "\nSelection Type : "
                 << (takenWeight < item.weight ? "Partial" : "Full");
            cout << "\n-----------------------------------------\n";
        }
    }

    cout << "\nTotal Items Selected: " << selectedCount << endl;

    return totalValue;
}

int main() {
    vector<Item> items = {
        Item("Medical Kits", 10, 100, false, 1),
        Item("Food Packets", 20, 60, true, 3),
        Item("Drinking Water", 30, 90, true, 2),
        Item("Blankets", 15, 45, false, 3),
        Item("Infant Formula", 5, 50, false, 1)
    };

    double capacity;

    cout << "=======================================\n";
    cout << "     EMERGENCY RELIEF SUPPLY SYSTEM\n";
    cout << "=======================================\n";

    cout << "\nPriority Levels: 1 = High, 2 = Medium, 3 = Low\n";
    cout << "Enter maximum boat capacity (kg): ";
    cin >> capacity;
```

```
    if (cin.fail() || capacity <= 0) {
        cout << "Invalid capacity. Enter a positive number.\n";
        return 1;
    }

    double totalWeight;
    double totalValue = fractionalKnapsack(items, capacity, totalWeight);

    cout << "\n========================================\n";
    cout << "               FINAL REPORT\n";
    cout << "========================================\n";

    cout << "Total Weight Carried : " << fixed << setprecision(2)
         << totalWeight << " kg\n";

    cout << "Remaining Capacity   : " << capacity - capacity + 0
         << " kg\n";

    cout << "Total Utility Value  : " << totalValue << " units\n";

    cout << "========================================\n";

    return 0;
}
```

### 10.2 Execution Output:

Below is the terminal execution output demonstrating optimal disaster relief supply allocation for boat capacity limit W = 40 kg:

```
om29dev@DESKTOP-MH6NVIC:~/daa$ vi emergency_relief_knapsack.cpp
om29dev@DESKTOP-MH6NVIC:~/daa$ g++ emergency_relief_knapsack.cpp -o emergency_relief_knapsack
om29dev@DESKTOP-MH6NVIC:~/daa$ ./emergency_relief_knapsack
========================================
   EMERGENCY RELIEF SUPPLY SYSTEM
========================================

Priority Levels: 1 = High, 2 = Medium, 3 = Low
Enter maximum boat capacity (kg): 40

========== SELECTED RELIEF ITEMS ==========

Item           : Medical Kits
Priority       : 1
Weight Taken   : 10.00 kg
Utility Value  : 100.00 units
Selection Type : Full
----------------------------------------
Item           : Infant Formula
Priority       : 1
Weight Taken   : 5.00 kg
Utility Value  : 50.00 units
Selection Type : Full
----------------------------------------
Item           : Drinking Water
Priority       : 2
Weight Taken   : 25.00 kg
Utility Value  : 75.00 units
Selection Type : Partial
----------------------------------------
Total Items Selected: 3
========================================
              FINAL REPORT
========================================
Total Weight Carried : 40.00 kg
Total Utility Value  : 225.00 units
========================================
om29dev@DESKTOP-MH6NVIC:~/daa$
```

*Figure 10.1: Emergency Relief Supply Allocation Execution Output*

## 11. Conclusion:

The Fractional Knapsack Algorithm maximizes utility by prioritizing items with the highest value-to-weight ratio. In emergency relief logistics, this ensures that life-saving items like medicine kits are transported first, followed by food and water as space allows. The greedy algorithm is both efficient (O(n log n)) and optimal for fractional cases, making it highly effective in disaster management where quick, resource-optimized decisions are critical.